3-Day Hands-On Workshop

Containers, Docker Compose, and Containerizing webMethods

Docker / Podman & Compose with IBM webMethods Microservices Runtime 12.1, from fundamentals to a production-shaped Swisscom landscape

3 days · 6 blocks · 6 capstones

Developers · Architects · DevOps

David Poland · Nov. 16, 2026

Workshop at a Glance

Day 1	Block 1 · Container fundamentals (Capstone 1: a database stack by hand) Block 2 · Your first webMethods container (Capstone 2: MSR as code)
Day 2	Block 3 · Building images & Compose (Capstone 3: one compose.yaml) Block 4 · webMethods stacks: MSR + UM + PostgreSQL (Capstone 4)
Day 3	Block 5 · The Swisscom shape: 400+ packages, CI, 2 × MSR, API Gateway (Capstone 5) Block 6 · Production readiness: secrets, rolling update, Graylog, migration (Capstone 6)
Format	Every 4-hour block: concepts → small exercises → one capstone the next block builds on Reference solutions for every capstone under blockN/solution/

Generic containers first, webMethods next, the Swisscom shape last.

Classification - Internal

Workshop Objectives

- Build in-depth, hands-on know-how for running webMethods in containers
- Get fluent with Docker/Podman Compose as a realistic target environment for multi-component stacks
- Internalize best practices for webMethods container deployments, and the anti-patterns to avoid
- Understand the Dos and Don'ts for production container environments
- Leave with a concrete Windows → Linux container migration strategy for Swisscom
- Work hands-on against a scaled-down version of the existing Swisscom architecture

Classification - Internal

Ground Rules & Logistics

How we'll work

- 09:00–17:00 with a lunch break; remote format runs as 4 h slots
- Short breaks roughly every 90 minutes
- Exercises and capstones are individual or in pairs; work at your own pace
- Use the shared chat for questions that don't block you
- A parking-lot list captures Swisscom-specific tangents for the Day 3 wrap-up

What you'll leave with

- This deck and the Participant Workbook
- The workshop repository: starters, reference solutions, scripts, runbook, CI example
- A Dos-and-Don'ts checklist you can apply immediately
- A first-draft migration roadmap outline for Swisscom
- Post-workshop follow-up by IBM Expert Labs (Day 5)

Classification - Internal

Your Environment for This Workshop

Required tooling

- Podman Desktop (or Docker Desktop) with Compose v2.24+
- VS Code + Git Bash
- IBM webMethods Service Designer 12.x (optional for the exercises)
- Access to the Swisscom GitLab and to the workshop repository
- Login to ibmwebmethods.azurecr.io: IBMid + registry token from containers.webmethods.io

Sizing & access

- ≥ 16 GB RAM, ≥ 50 GB free disk on the laptop
- Podman machine: give it ≥ 10 GB RAM for Capstones 5–6 (podman machine set --memory 10240)
- docker and podman commands are interchangeable; we flag the few differences
- Rootless by default: ports < 1024 and bind-mount ownership behave differently

If anything above isn't working by 09:30 on Day 1, flag it immediately.

Classification - Internal

The Workshop Repository

Git Repo: https://github.com/david-poland-IBM/sc-training.git

Folder	Content
packages/SwisscomLab	Sample IS package used by blocks 2–6: health, JDBC (orders) and JMS services, Java sources included
block1/	exercises/site · capstone: database stack by hand · solution/capstone1-solution.sh
block2/	commands only · capstone: application.properties TODOs · solution/capstone2-solution.sh
block3/	exercises: my-web, layer-order, entrypoint, multistage, msr-custom · capstone: one compose.yaml
block4/	exercises: um-only, msr-um · capstone: MSR + UM + PostgreSQL · solution incl. verify.sh
block5/	capstone: 2 × MSR, nginx, API Gateway, prod overlay, Graylog · solution incl. runbook, CI
block6/	exercises: overlays, secrets · Capstone 6 continues in block5/capstone

All images come from public registries: ibmwebmethods.azurecr.io for IBM products, Docker Hub for the rest.

Classification - Internal

The Swisscom Landscape We're Working Towards

- ~400 webMethods packages spread across a multi-component integration platform
- 2-instance Integration Server architecture, clustered per environment
- 4 environments: Dev, Test-internal, Test-external, Prod
- Universal Messaging as the target broker replacement
- API Gateway with custom packages for traffic filtering / policy enforcement
- Trading Networks and Active Transfer for B2B and managed file transfer
- Existing GitLab CI/CD pipelines and Ansible playbooks to integrate with
- Graylog for log aggregation, Nagios/OP5 for monitoring and alerting
- End goal: migrate the runtime platform from Windows to Linux containers

> **Note:** Every capstone is a deliberately scaled-down version of this architecture: small enough for a laptop, shaped enough to transfer directly.

Classification - Internal

Six Blocks, One Story

Block 1	Block 2	Block 3	Block 4	Block 5	Block 6
Container fundamentals	First webMethods container	Building images & Compose	MSR + UM + PostgreSQL	The Swisscom shape	Production readiness

Each capstone is the starting point of the next block: a database by hand → one MSR configured as code → one compose.yaml → a 3-service webMethods stack → 2 × MSR behind API Gateway → secrets, rolling update, Graylog.

Classification - Internal

How Every 4-Hour Block Works

Concepts	Short slide sessions, 15–30 min each, right before you need them
🔧 **Exercises**	Small, single-skill exercises (5–20 min) with ✏️ questions in the Participant Workbook
🏆 **Capstone**	One deliverable per block that combines everything, with a reference solution
🏁 **Checkpoint**	Recap questions and discussion before the next block
Breaks	15 min in the middle of every block; remote: two 4-hour slots per day

Workshop material: one folder per block, each with exercises/, capstone/ and solution/. Capstones 5–6 lives in block5/ and continues in Block 6.

Classification - Internal

Day 1

Fundamentals & Your First webMethods Container

What a container really is, then IBM webMethods Microservices Runtime 12.1 in a container

Block 1 · Container fundamentals

Block 2 · Your first webMethods container

Day 1: Agenda

Block 1 · 09:00–13:00

- Welcome, objectives, the Swisscom landscape
- Why containers? Concepts and lifecycle
- What is a container, technically?
- Namespaces · cgroups · union FS
- Podman vs. Docker, essential commands
- Volumes and networks
- Container security, Trivy
- 🏆 A database stack by hand

Block 2 · 13:00–17:00

- IBM webMethods container registry
- Integration Server vs. MSR, the MSR image
- Environment variables, health & metrics, logs
- application.properties: configuration as code
- Service Designer
- 🏆 Capstone 2: first MSR, configured as code
- Q&A, Day 1 recap

Hands-on: *Exercises 1.1–1.8 and 2.1–2.4 · Capstone 1 (30 min) · Capstone 2 (80 min)*

Classification - Internal



Block 1

Container Fundamentals

Concepts · Linux primitives · Podman · Volumes · Networks · Security



Why Containers?

The shipping container revolution — applied to software

The Shipping Container Analogy

Before Containers

- Different sizes, shapes, formats
- "Works on my machine" failures
- Dependency conflicts between apps
- Environment drift: dev ≠ staging ≠ prod
- Manual server provisioning

With Containers

- Standard packaging format (OCI)
- Runs identically everywhere
- Isolated dependencies per container
- Same image across all environments
- Automated, declarative deployment

Classification - Internal

The Container Ecosystem

BUILD	SHIP	RUN	ORCHESTRATE
Containerfile / Dockerfile podman build docker build	Container Registry DockerHub, Harbor, ACR, ECR, GHCR	Container Runtime Podman, containerd, CRI-O	Kubernetes / OpenShift Scheduling, Scaling, Self-healing

Classification - Internal

Core Container Concepts

Image	Read-only template with filesystem, binaries, and config. Built in layers.
Container	Running instance of an image with a thin writable layer on top.
Registry	Repository for storing and distributing images (DockerHub, Harbor, ACR).
Runtime	Software that creates and runs containers (containerd, CRI-O, Podman).
Layer	Each Containerfile instruction creates a cacheable, shareable filesystem layer.

Classification - Internal

Container Lifecycle

Created	Running	Stopped	Removed
Image pulled, writable layer allocated, no process running	Main process (PID 1) is executing	Process terminated. Can restart or inspect.	Container and writable layer deleted. Use volumes to persist.

create → start → stop / crash → rm

Classification - Internal



What Is a Container, Technically?

It’s not a VM. It’s an isolated process on a shared kernel.

A Container Is a Process

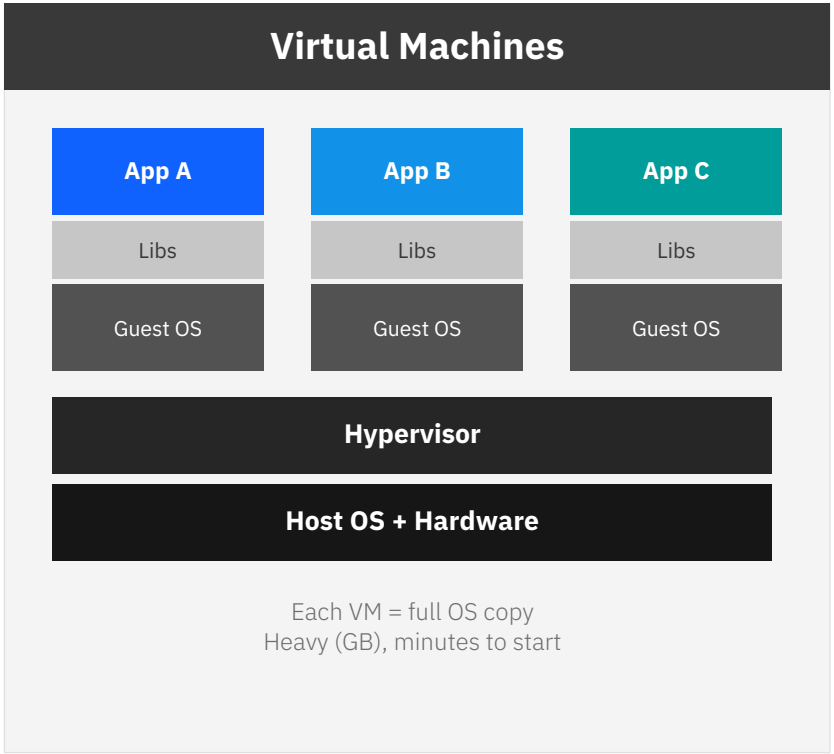
A container is a process (or group of processes) running on the host OS, isolated from other processes using kernel features.

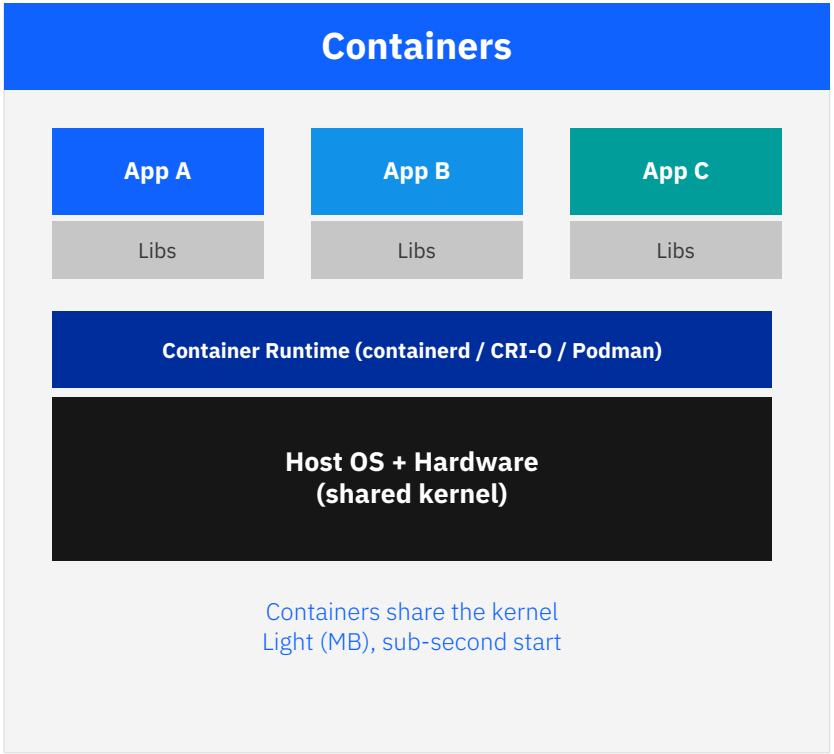
It does NOT have its own kernel. It shares the host kernel.

Isolation	Resource Control	Layered Filesystem
Each container has its own view of the system: processes, network, filesystem	CPU, memory, I/O are capped per container. No noisy neighbors.	Read-only image layers + thin writable layer. Efficient and shareable.

Classification - Internal

VMs vs. Containers — Architecture





Classification - Internal

VMs vs. Containers — Comparison

Aspect	Virtual Machine	Container
Startup time	Minutes	**Sub-second to seconds**
Image size	Gigabytes	**Megabytes**
Isolation	Full OS kernel per VM	Process-level (shared kernel)
Overhead	High — each VM has its own OS	**Low — shared host kernel**
Portability	Hypervisor-dependent	Any OCI-compatible runtime
Density per host	Tens of VMs	**Hundreds of containers**
Security	**Strong — separate kernel**	Good — kernel shared, namespace-isolated

💬 *Discussion: When would you still choose a VM over a container?*

Classification - Internal

What Makes Container Isolation Work?

Three Linux kernel features combine to create what we call a “container”:

1

Namespaces

Isolation

Give each container its own view of the system — its own process IDs, network stack, filesystem mounts, hostname.

2

Cgroups

Resource Control

Limit and account for CPU, memory, I/O, and network usage per container. Prevent noisy neighbors.

3

Union Filesystem

Efficient Storage

Stack read-only image layers with a thin writable layer on top. Base layers are shared across containers.

Classification - Internal



Linux Primitives Behind Containers

Namespaces · Cgroups · Union Filesystems — The Apartment Building

The Apartment Building Analogy

The Host Operating System = The Apartment Building

Each container is an apartment in the building. They share the foundation (kernel) but are separated from each other.

Namespaces

The walls between apartments

Each apartment has its own address, door numbers, and mailbox. You can't see into your neighbor's apartment.

Cgroups

The utility meters

Ensure no apartment hogs all the electricity (CPU), water (memory), or elevator capacity (I/O). The landlord sets limits.

Union FS

The apartment floor plan

The building provides an initial floor plan (read-only layers). You can add your own decorations (writable layer), but the floor plan is shared.

Classification - Internal

Namespaces — Isolation Between Containers

PID — **Process IDs**

Container sees only its own processes. PID 1 is its main process, not the host's init.

NET — **Network**

Own IP address, ports, routing table. Container's port 80 doesn't conflict with another container's.

MNT — **Mount Points**

Own filesystem view. Container sees its image layers, not the host's files.

UTS — **Hostname**

Own hostname. Allows each container to have a unique identity.

USER — **User / Group IDs**

Maps container UIDs to different host UIDs. Root inside ≠ root on host (rootless).

IPC — **Inter-Process Comm.**

Isolated shared memory and message queues. Prevents cross-container data leaks.

Classification - Internal

Namespaces — What Each Container Sees

Host View (Reality)

```
Processes:  PID 1 (init), PID 204 (sshd),
            PID 1052 (nginx), PID 1053 (node),
            PID 1099 (postgres) ...

Network:    eth0: 192.168.1.100
            All ports globally shared

Filesystem: / (full host filesystem)

Hostname:   prod-server-01

Users:      root (0), ubuntu (1000), ...
```

Container View (Namespaced)

```
Processes:  PID 1 (nginx) ← only mine!
            PID 2 (nginx worker)
            (that's it — I can't see others)

Network:    eth0: 172.17.0.2
            My own port 80 — no conflict

Filesystem: / (image layers + writable)
            /host files invisible

Hostname:   a3f7b2c91d04

Users:      app (1000) → mapped to 100000 on host
```

Each container believes it has the entire machine to itself. Namespaces create this illusion.

Classification - Internal

Cgroups — Resource Limits per Container

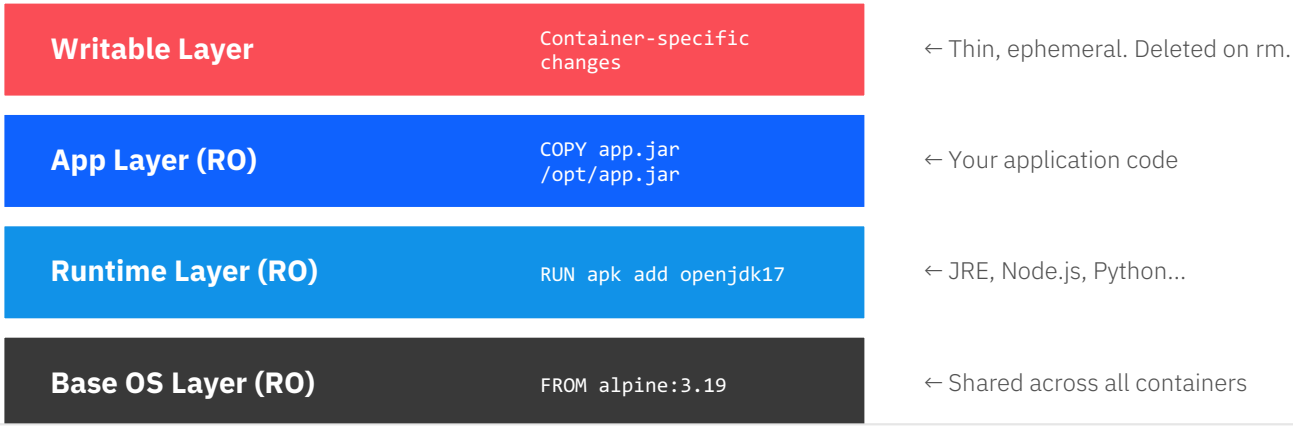
Without cgroups: A single container could consume all CPU and memory, crashing its neighbors.
With cgroups: The “landlord” sets metered limits per tenant.

CPU	Limit CPU time (e.g., 0.5 cores). Container cannot use more than its share.	`--cpus 0.5`
Memory	Hard cap on RAM. If exceeded, container is OOM-killed by the kernel.	`--memory 512m`
Block I/O	Throttle disk read/write speed. Prevents I/O-heavy containers from saturating disk.	`--blkio-weight 500`
Network	Limit network bandwidth per container (via tc or cgroup net_cls).	`net_cls cgroup`

⚠ *For webMethods MSR: Always use -XX:+UseContainerSupport so the JVM respects cgroup memory limits*

Classification - Internal

Union Filesystem — Layered Image Architecture



Copy-on-Write — When a container modifies a file from a read-only layer, it copies the file up to the writable layer first. The original layer is untouched.

Layer Sharing — If 10 containers use the same base image, the base layer is stored only once on disk. Massive space savings.

Cache Efficiency — When rebuilding, unchanged layers are cached. Only modified layers are rebuilt. Order instructions from stable → volatile.

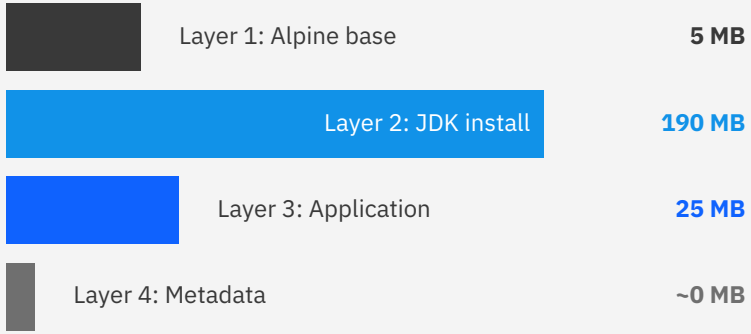
Classification - Internal

Image Layers in Practice

Containerfile

```
FROM alpine:3.19
RUN apk add openjdk17
COPY app.jar /opt/
CMD ["java","-jar","app.jar"]
```

Resulting Layers



Total image: ~220 MB. If you change only app.jar, only Layer 3 is rebuilt!

This is why Containerfile instruction order matters: put stable layers first, changing layers last.

Classification - Internal

Summary — The Three Pillars of Containers

1 **Namespaces**

Isolation

Each container gets its own PID, NET, MNT, UTS, USER, IPC view of the system

🧱 Walls between apartments

2 **Cgroups**

Resource Control

CPU, memory, I/O, network are metered and capped per container

📊 Utility meters in the building

3 **Union FS**

Efficient Storage

Read-only image layers + writable layer. Copy-on-write. Shared base layers.

🏠 Common apartments floor plan (shared layout)

Together, these three kernel features create the lightweight, portable, isolated units we call containers.

Classification - Internal

🔧 Hands-On: Exercises 1.1–1.2

1.1	Verify your engine: podman, podman-compose, podman machine, hello-world
1.2	Inside a container: ps, hostname, whoami; then the same process from the host (podman top, HPID)
📁 **Files**	No files needed: alpine:3.19 and hello-world from Docker Hub

Participant Workbook, Block 1 · ~20 min · 🏁 sleep is PID 1 inside, an ordinary PID on the host

Classification - Internal

🔧 Hands-On: Exercises 1.3–1.4

1.3	nginx: -p 8080:80, logs, exec, podman cp a new page, podman diff; re-create: the change is gone
1.4	-e and --env-file; --memory/--cpus become cgroup files; tail /dev/zero under 64 MB → OOM, exit 137
📁 Files	No files needed: nginx:1.27-alpine, alpine:3.19

Participant Workbook, Block 1 · ~25 min · 🏁 memory.max = 134217728; OOMKilled=true, exit 137

Classification - Internal

Essential Container Commands

Category	Command	Purpose
Image	podman build -t app:1.0 .	Build from Containerfile
Image	podman push registry/app:1.0	Push to registry
Run	podman run -d -p 8080:80 nginx	Start container (detached, port map)
Lifecycle	podman stop / start / restart	Manage container state
Debug	podman logs -f / podman exec -it	Follow logs / shell into container
Info	podman inspect <name>	Detailed metadata (JSON)
Cleanup	podman system prune -a	Remove all unused resources

All docker commands work identically with podman — just swap the binary name.

Classification - Internal

Configuration: Volumes, Networks, Env Vars

Volumes

Persist data beyond
container lifecycle.

Named: -v data:/app/data
Bind: -v ./host:/data
tmpfs: --tmpfs /tmp

Analogy: USB drive

Networks

Service discovery
by container name.

podman network create net
podman run --network net
  --name backend ...

Containers resolve names

Env Vars

Inject configuration
at runtime.

-e KEY=VALUE
--env-file app.env

Mount config files:
-v ./conf:/etc/app:ro

Classification - Internal

Networking Basics

- bridge (default): a private virtual network; each container gets its own IP
  - User-defined networks add DNS: containers reach each other by name (um, postgres, msr-1)
- host: shares the host network namespace (no isolation, no port mapping)
- Publishing: -p HOST:CONTAINER, e.g. -p 5555:5555 or -p 15555:5555 for a second instance
- Without -p a port is only reachable from containers on the same network
- Compose (Day 2) creates one network per project and registers every service name in DNS

Gotcha: Rootless Podman cannot bind host ports < 1024 by default. Map to a high port instead.

Classification - Internal

Volume Management

- Named volumes: engine-managed, survive podman rm, deleted only by podman volume rm (or compose down -v)
- Bind mounts: an exact host path, ideal for config files under Git (application.properties)
- tmpfs: in memory only, gone on stop
- -v msr-logs:/opt/webmethods/IntegrationServer/logs → named volume
- -v $PWD/application.properties:/opt/webmethods/IntegrationServer/application.properties:ro → bind mount
- Ownership matters: MSR runs as UID 1724; a host folder on Linux must be writable for 1724

Gotcha: For MSR: logs, externalized packages and the UM data store go on volumes. The image itself stays immutable.

Classification - Internal

🔧 Hands-On: Exercises 1.5–1.7

1.5	Lifecycle and essential commands: stats, inspect --format, port, top, stop/start/restart, cp, system df
1.6	Named volume survives re-create; bind mount edits show up live
1.7	Default network: 'bad address backend'. User-defined network: resolves by name
📁 Files	block1/exercises/site (bind mount in 1.6)

Participant Workbook, Block 1 · ~30 min · 🏁 data survived; backend resolved on workshop-net only

Classification - Internal



Container Security

Image · Runtime · Network · Host — Defense in depth

Container Security Layers

Image Security	Minimal base images · Vulnerability scanning (Trivy, Grype) · Image signing · Pin digests · No embedded secrets
Runtime Security	Non-root user (USER directive) · Drop capabilities · Read-only rootfs · Resource limits · No privileged mode
Network Security	Limit exposed ports · Network policies in Kubernetes · TLS/mTLS encryption
Host Security	Patched host OS · SELinux/AppArmor · Rootless containers (Podman default) · Limit host mounts

Golden Rule: Always run as non-root. In webMethods MSR, use sagadmin (UID 1724).

Classification - Internal

🔧 Hands-On: Exercise 1.8

1.8	Trivy as a container: scan alpine:3.19 and nginx:1.27-alpine for HIGH/CRITICAL findings
📁 Files	docker.io/aquasec/trivy:0.56.2 + volume trivy-cache (kept for Block 5)

Participant Workbook, Block 1 · ~10 min · 🏁 nginx has many more findings than alpine; numbers change daily

Classification - Internal

🏆 Capstone 1: A Database Stack by Hand

Objectives

30 min

- Network shop-net and named volume shop-data
- shop-db from postgres:16: user/db shop, password of your choice
- Volume on /var/lib/postgresql/data, --memory 512m --cpus 1
- Healthcheck pg_isready; wait until healthy
- Throw-away psql container on shop-net reaches "shop-db" by name: create table orders, insert 2 rows
- podman rm -f shop-db, start it again: the 2 rows are still there
- podman stats shows the limits; a cleanup argument removes everything

Prerequisites

- Exercises 1.1–1.8 done
- postgres:16 and alpine:3.19 pulled
- Reference: block1/solution/capstone1-solution.sh

Deliverable
capstone1.sh + answers: flags needed, what happens without the volume, why the name resolves. Keep it: Block 3 replaces it with one compose.yaml.

Classification - Internal

💬 Q&A and Discussion: Capstone 1

1. *Which flags of your shop-db command would differ between Dev and Prod, and who should own them?*

2. *Without the named volume, what would step 5 have lost, and what is the 'shop-data' of today's IS servers?*

3. *Why did shop-db resolve by name on shop-net, and what does that mean for hostnames and IPs in today's configs?*

Classification - Internal

Block 1 Recap: Container Fundamentals

What We Covered

✓ 1.1–1.2 Engine verified; a container is a process
PID 1 inside, a normal host PID outside (podman top)

✓ 1.3–1.4 Ports, logs, exec; cgroup limits, OOM = 137
The writable layer vanishes on re-create

✓ 1.5–1.7 Lifecycle, volumes, bind mounts, networks
Names resolve only on user-defined networks

✓ 1.8 + 🏆 Trivy scan; a database stack by hand

Key Concepts Learned

- Namespaces isolate, cgroups limit, layers share
- Container lifecycle: create → start → stop → rm
- Data on volumes, configuration from outside
- Service discovery by name on a shared network
- Exit code 137 = SIGKILL (OOM or stop timeout)
- Scan images; new CVEs appear every day

Next: Block 2, your first webMethods container

Classification - Internal